

基于 MWORKS.Syslab 的 Floyd 并行算法实现与优化

一、案例简介与背景

1.1 问题背景：全源最短路径（All-Pairs Shortest Paths, ASP）

在图论中，全源最短路径问题是一个基础且重要的问题。其目标是找到一个图中任意两个节点之间的最短路径长度。该问题在许多领域都有广泛应用，例如：

- **交通网络规划**：计算城市中任意两点间的最快到达路径。
- **计算机网络**：确定网络中数据包传输的最佳路由。
- **生物信息学**：分析分子间的相互作用距离。
- **社交网络分析**：度量网络中个体之间的紧密程度。

Floyd-Warshall 算法是解决 ASP 问题的一种经典动态规划算法。它通过迭代的方式，逐步允许更多的中间节点来优化路径长度，最终得到所有节点对之间的最短距离。

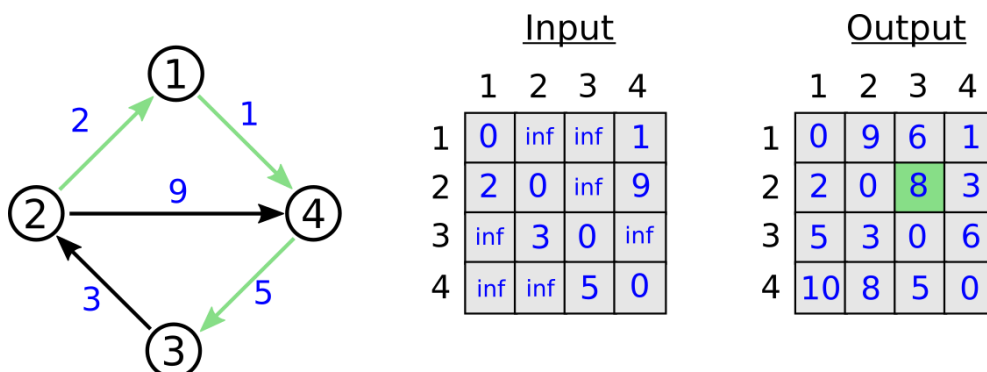
1.2 工程挑战：并行计算的需求

随着数据规模的急剧增长，串行算法在处理大规模 ASP 问题时往往面临性能瓶颈。并行计算通过利用多个处理器协同工作，为加速这类计算密集型任务提供了有效途径。然而，并行算法的设计与实现并非易事，需要仔细考虑数据划分、任务分配、处理器间通信以及同步等问题。

本案例将引导同学们深入研究 Floyd-Warshall 算法的并行化过程，使用消息传递接口（Message Passing Interface, MPI）这一广泛应用于高性能计算领域的并行编程模型。学生将亲身体验从串行算法分析、并行策略设计、MPI 编程实现、性能瓶颈识别到最终优化验证的完整工程实践流程。

二、案例描述

本案例的核心任务是设计、实现并优化一个基于 MPI 的并行 Floyd-Warshall 算法。我们将以一个带权有向图作为输入，该图的连接关系和权重通过一个距离矩阵 C 表示。如果节点 i 到节点 j 没有直接连接，则对应矩阵元素 $C[i,j]$ 为一个极大值（代表无穷大）。算法的目标是计算出一个新的距离矩阵，其中每个元素表示对应节点对之间的最短路径长度。



Floyd-Warshall 串行算法的核心思想是通过 k 次迭代，在第 k 次迭代中，允许路径经过节点 1 到 k ，并更新任意节点 i 到 j 的最短路径。其核心更新公式为：

$$C[i,j] = \min(C[i,j], C[i,k] + C[k,j])$$

我们将采用行划分的策略进行并行化：将距离矩阵 C 的行分配给不同的 MPI 进程。每个进程负责更新其分配到的那些行对应的路径信息。在每次迭代（ k 循环）中，持有第 k 行数据的进程需要将该行广播给所有其他进程，因为所有进程在更新自己的数据时都需要用到第 k 行和第 k 列的数据。

案例将特别关注并行实现中可能出现的通信同步问题，并引导学生通过改进通信方式（例如，使用 `MPI_Bcast!` 替代点对点发送，或者确保接收顺序）来保证算法的正确性。

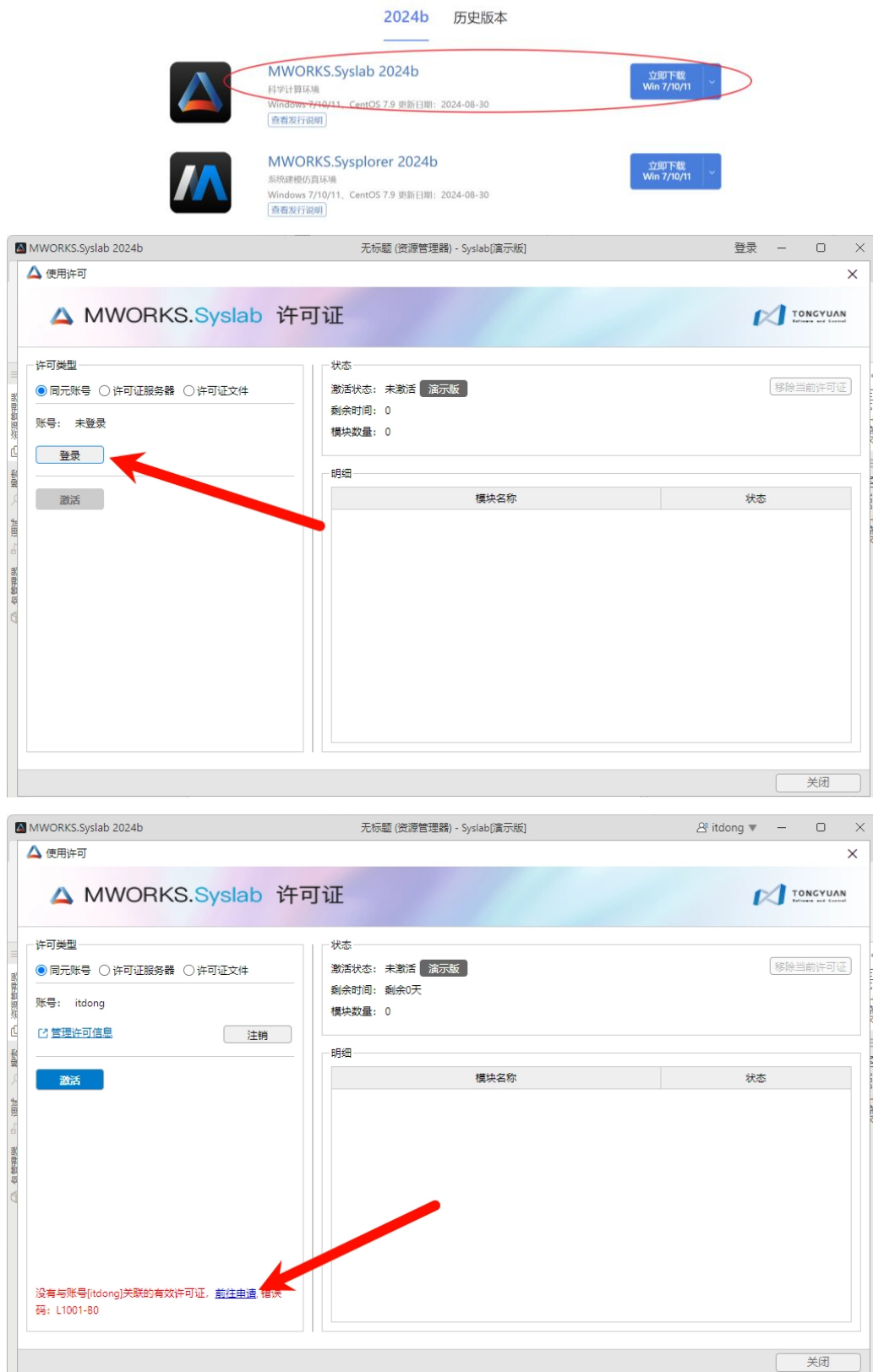
三、实验环境与设置

硬件：多核处理器或计算机集群，支持 MPI 并行环境。

软件：

1. 操作系统：Windows 7/10/11 x64。
2. 编程语言：Julia (及 MPI.jl 包)。

3. 集成开发环境：MWORKS.Syslab (<https://www.tongyuan.cc/download>)



许可申请


MWORKS.Sysplorer
系统建模仿真环境


MWORKS.Syslab
科学计算环境

个人版

支持账号授权
允许绑定3台设备
有效期3个月
仅限个人非商业用途使用

立即申请
联系销售

教育版

支持账号授权
允许绑定3台设备
有效期6个月
仅限教育邮箱申请

立即申请
联系销售

企业版

支持账号授权与文件授权
允许绑定3台设备
试用有效期1个月
提供1次试用机会
企业版用于广泛的商业用途

申请试用
联系销售

*最终解释权归苏州同元软控信息技术有限公司所有

功能模块	个人版	教育版	企业版
交互式编程环境	✓	✓	✓
解释与调试模块	✓	✓	✓

MWORKS.Syslab 2024b
无标题 (资源管理器) - Syslab(教育版)
itdong


许可证


许可类型

☒ 同元账号 ☐ 许可证服务器 ☐ 许可证文件

账号: itdong

[管理许可信息](#) 注销

激活

状态

激活状态: 账号许可 教育版 移除当前许可证

剩余时间: 剩余180天

模块数量: 23

明细

	模块名称	状态
1	MW_Syslab_Standard	剩余180天
2	MW_Syslab_ExternalInterfaces	剩余180天
3	MW_Syslab_MImport	剩余180天
4	MW_Syslab_TySymbolicMath	剩余180天
5	MW_Syslab_TyCurveFitting	剩余180天
6	MW_Syslab_TySignalProcessing	剩余180天
7	MW_Syslab_TyCommunication	剩余180天
8	MW_Syslab_TyDSPSystem	剩余180天
9	MW_Syslab_TyControlSystems	剩余180天

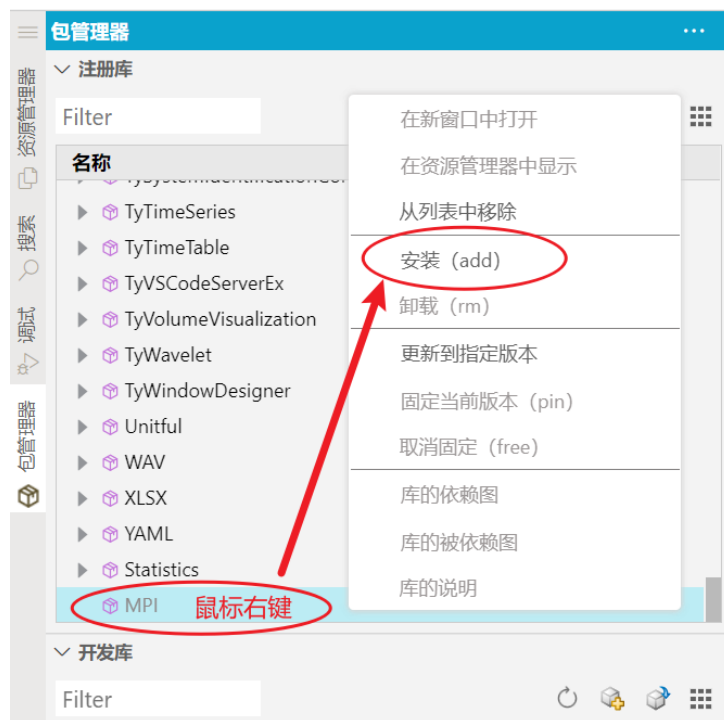
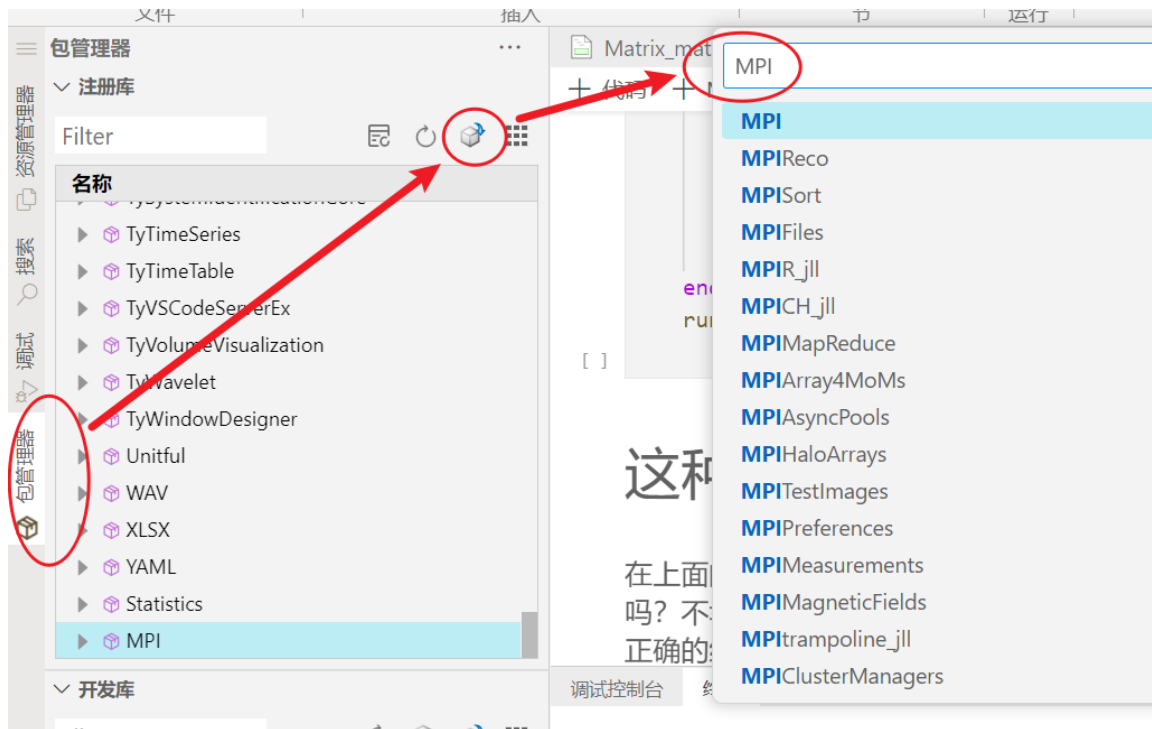
✓ 激活成功!
关闭

四、实验指导手册

4.1 准备工作：安装与配置

安装 **MWORKS.Syslab**: 从 <https://www.tongyuan.cc/download> 下载并安装 MWORKS.Syslab。

安装 **MPI**:



4.2 理解和实现串行 Floyd-Warshall 算法

1. 理解算法逻辑

仔细阅读关于 Floyd 串行算法的描述，理解三层嵌套循环的含义，特别是最外层 k 循环的作用。

思考：为什么 k 循环必须在最外层？（提示：迭代 k 保证了在考虑节点 k 作为中间节点时， $C[i,k]$ 和 $C[k,j]$ 已经是只允许通过节点 1 到 $k-1$ 的最短路径）。

2. 分析或实现串行代码

`floyd!(C)`函数的 Julia 实现如下：

```
function floyd!(C)
    n = size(C, 1)
    @assert size(C, 2) == n
    for k in 1:n
        for j in 1:n
            for i in 1:n
                @inbounds C[i, j] = min(C[i, j], C[i, k] +
C[k, j])
            end
        end
    end
    C
end
```

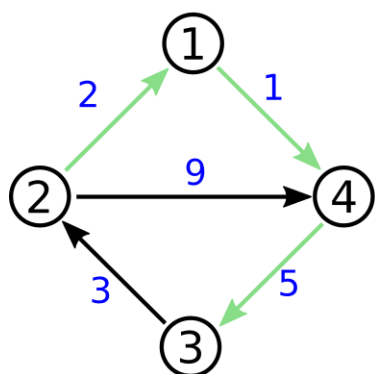
注意：`floyd!`函数实现中，中间循环是 j ，最内层循环是 i 。Julia 是列主序存储，因此 `floyd!`的访问模式（ $C[i,j]$ 在最内层循环中 i 变化快）更高效。

3. 测试串行代码

使用如下示例图和输入矩阵进行测试。

```
inf = 1000
C = [
    0 inf inf 1
    2 0 inf 9
    inf 3 0 inf
    inf inf 5 0
]
floyd!(C)
```

4. 预期输出



Input				
	1	2	3	4
1	0	inf	inf	1
2	2	0	inf	9
3	inf	3	0	inf
4	inf	inf	5	0

Output				
	1	2	3	4
1	0	9	6	1
2	2	0	8	3
3	5	3	0	6
4	10	8	5	0

验证得到的输出是否与预期输出一致。

4.3 分析数据依赖性与并行化策略

4.3.1 并行化策略

出于性能原因，我们采用较大的并行化粒度，采用与矩阵-矩阵乘积类似的并行策略：

每个进程将在每次迭代 k 中更新距离表 C 的一部分连续行。

Distance matrix C

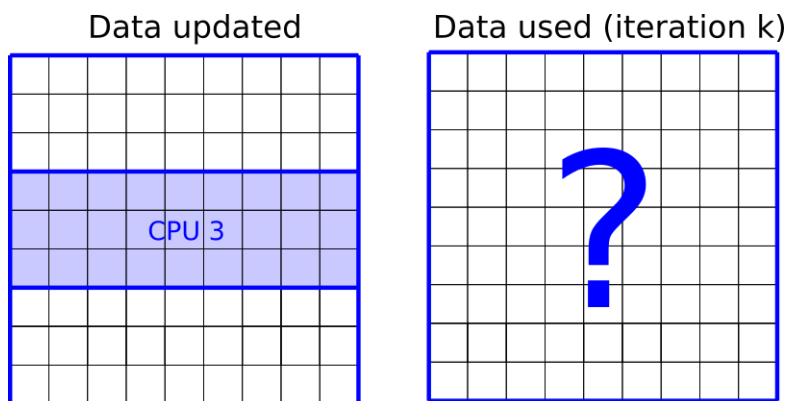
CPU 1									
CPU 2									
CPU 3									

4.3.2 数据依赖性

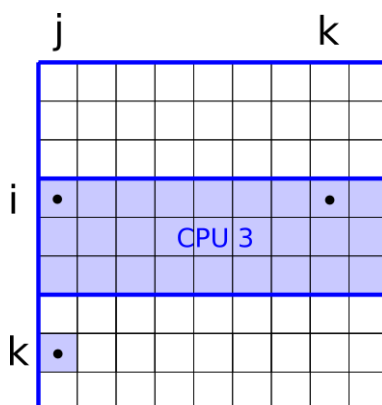
回顾：我们在迭代 k 执行此更新：

$$C[i,j] = \min(C[i,j], C[i,k] + C[k,j])$$

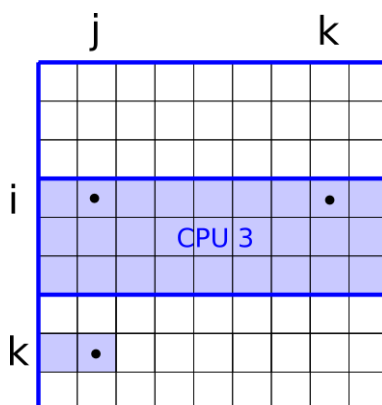
如果每个进程更新矩阵 C 的一个行块，那么这个操作需要哪些数据？



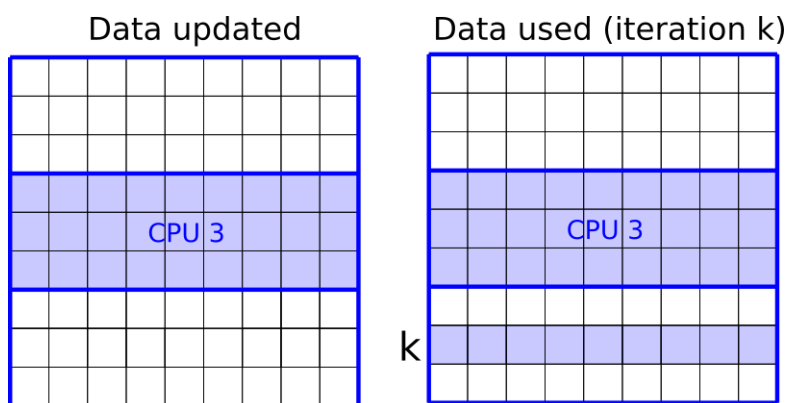
在每次迭代中，处理器需要输入值 $C[i,j]$ 、 $C[i,k]$ 和 $C[k,j]$ 。由于我们按行分割数据，进程已经拥有值 $C[i,j]$ 和 $C[i,k]$ 。然而， $C[k,j]$ 可能存储在不同的进程中，见下图。



当我们遍历列 j 时，进程需要来自第 k 行所有值作为输入。因此，在迭代 k 开始时，整个第 k 行需要被通信。



总而言之，在迭代 k ，一个给定的进程将需要第 k 行，而该行可能远程存储在另一个进程中。第 k 行的拥有者需要在迭代 k 将该行广播给其他进程。



通信需求:

在第 k 次迭代开始时, 拥有第 k 行 (我们称之为 row_k) 的那个进程, 必须将 row_k 的数据广播给所有其他进程。因为所有进程在更新它们各自负责的 $C[i,j]$ 时, 都需要 $C[k,j]$ 的值, 而这些值就构成了 row_k 。

4.4 实现初步的并行 Floyd-Warshall 算法 (基于 MPI)

本部分将实现并行 Floyd 算法, 包括输入分发、并行迭代计算和结果收集。我们将首先实现一个版本, 该版本在 `floyd_iterations!` 函数中使用点对点通信 (`MPI.Send` 和 `MPI.Recv!`) 来分发第 k 行, 这可能存在同步问题, 正如后续所讨论的那样。

4.4.1 初始化 MPI 环境和获取基本信息

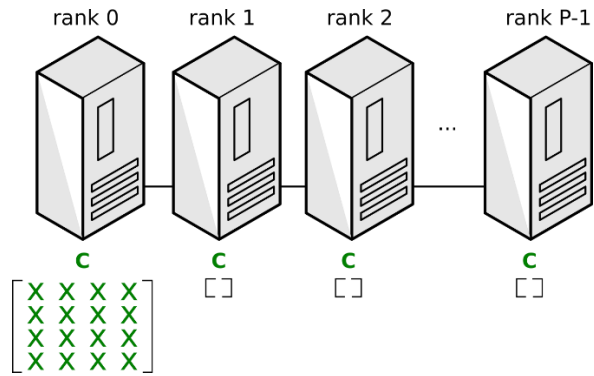
我们将使用以下函数来实现并行化的 Floyd 算法:

`floyd_mpi!(C, comm)`

该函数与串行函数 `floyd!(C)` 相似, 但有一些重要区别:

此函数将在多个 MPI rank 上调用, 但只有一个 rank (在下面的实现中为 rank 0) 将接收输入矩阵, 其他 rank 将接收一个空矩阵。如果所有 rank 都接收输入矩阵的副本, 内存效率会很低, 因为输入矩阵可能非常大。

在输出时, 只有 rank 0 上的 C 值是正确的。尝试在所有 rank 上写入结果效率不高。并行函数接受一个 MPI 通信子对象。这允许用户决定使用哪个通信子。例如, 直接使用 `MPI.COMM_WORLD` 或其副本 (这是推荐的方法)。

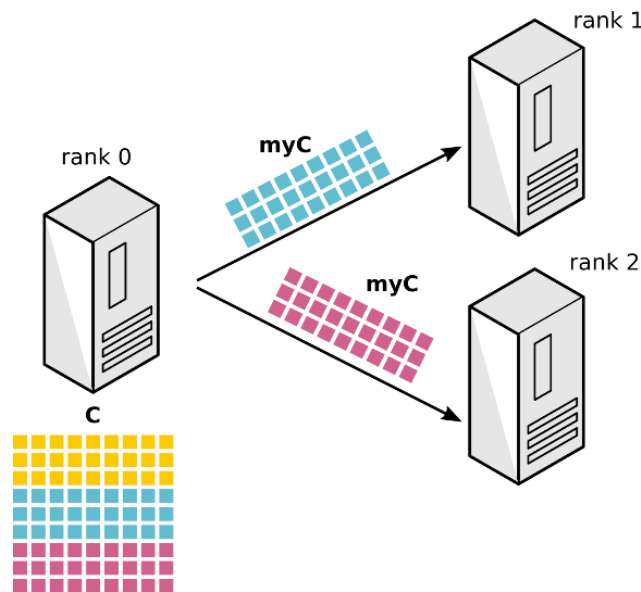


并行函数的实现在以下代码片段中给出：

```
function floyd_mpi!(C,comm)
    myC = distribute_input(C,comm)
    floyd_iterations!(myC,comm)
    collect_result!(C,myC,comm)
end
```

4.4.2 分发输入矩阵

由于只有 rank 0 接收输入矩阵 C ，该 rank 需要按行将其分割并将片段发送给所有其他 rank。这是在下面的函数中完成的。我们首先将问题规模 N 通信给所有 rank (开始时只有 rank 0 知道问题规模)。这可以通过 `MPI.Bcast!` 轻易完成。一旦所有 rank 都知道问题规模，它们就可以为其本地部分的 C (在代码中称为 `myC`) 分配空间。此后，rank 0 将片段发送给所有其他 rank。我们在这里使用 `MPI.Send` 和 `MPI.Recv!`。这也可以用 `MPI.Scatter!` 完成，但由于我们使用的是行分区并且 Julia 以列主序存储矩阵，因此更具挑战性。注意，该算法也可以使用列分区实现。在这种情况下，使用 `MPI.Scatter!` 可能是最佳选择。



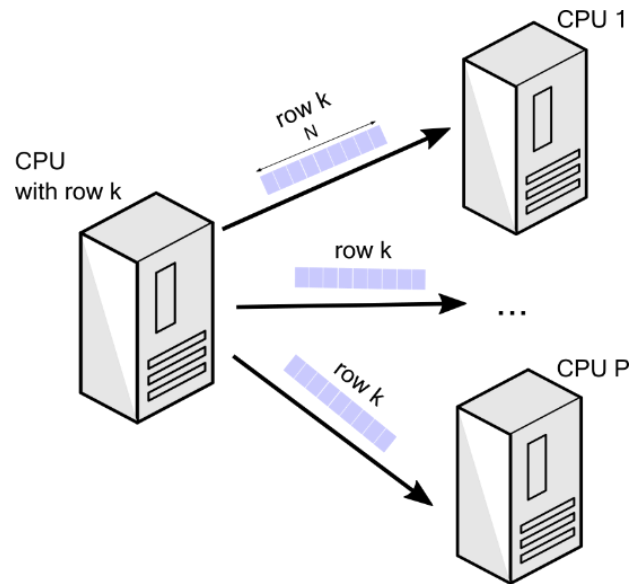
```

function distribute_input(C,comm)
    rank = MPI.Comm_rank(comm)
    P = MPI.Comm_size(comm)
    # Communicate problem size
    if rank == 0
        N = size(C,1)
        if mod(N,P) !=0
            println("N not multiple of P")
            MPI.Abort(comm,-1)
        end
        Nref = Ref(N)
    else
        Nref = Ref(0)
    end
    MPI.Bcast!(Nref,comm;root=0)
    N = Nref[]
    # Distribute C row-wise
    L = div(N,P)
    myC = similar(C,L,N)
    if rank == 0
        lb = L*rank+1
        ub = L*(rank+1)
        myC[:,:] = view(C,lb:ub,:)
        for dest in 1:(P-1)
            lb = L*dest+1
            ub = L*(dest+1)
            MPI.Send(view(C,lb:ub,:),comm;dest)
        end
    else
        source = 0
        MPI.Recv!(myC,comm;source)
    end
    return myC
end

```

4.4.3 并行运行 Floyd 更新

如上所述，我们需要通信矩阵 C 的第 k 行以便执行算法的第 k 次迭代。下面的函数与串行函数 `floyd!` 类似，但有一个关键区别。在迭代 k 开始时，第 k 行的拥有者将其发送给其他处理器。一旦第 k 行可用，所有 `rank` 都可以在其矩阵 C 的部分上本地执行 Floyd 更新。



```

function floyd_iterations!(myC,comm)
    L = size(myC,1)
    N = size(myC,2)
    rank = MPI.Comm_rank(comm)
    P = MPI.Comm_size(comm)
    lb = L*rank+1
    ub = L*(rank+1)
    C_k = similar(myC,N)
    for k in 1:N
        if (lb<=k) && (k<=ub)
            # Send row k to other workers if I have it
            myk = (k-lb)+1
            C_k[:] = view(myC,myk,:)
            for dest in 0:(P-1)
                if rank == dest
                    continue
                end
                MPI.Send(C_k,comm;dest)
            end
        else
            # Wait until row k is received
            MPI.Recv!(C_k,comm,source=MPI.ANY_SOURCE)
        end
        # Now, we have the data dependencies and
        # we can do the updates locally
        for j in 1:N
            for i in 1:L
                myC[i,j] = min(myC[i,j],myC[i,k]+C_k[j])
            end
        end
    end
end

```

```

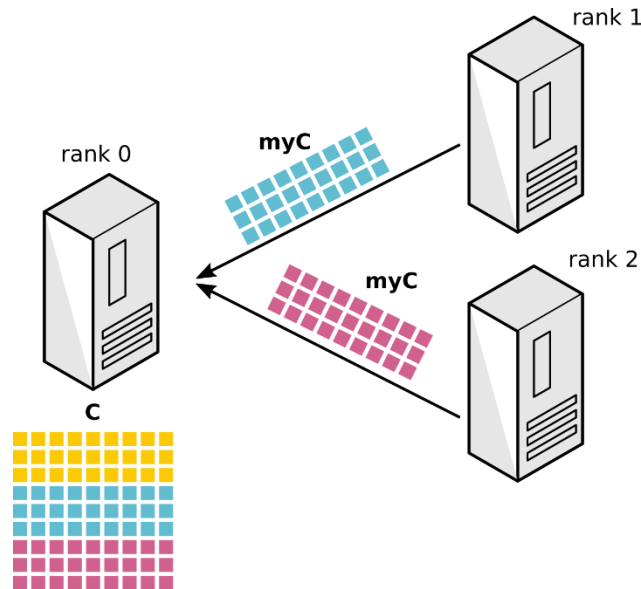
end
end
myC
end

```

4.4.4 收集结果

此时，我们已经解决了 ASP 问题，但解决方案被切割成不同的片段，每个片段存储在不同的 MPI rank 上。

通常将解决方案收集到单个矩阵中是很有用的，例如，将其与串行算法进行比较。以下函数收集所有片段并将它们存储在 rank 0 的 C 中。同样，我们用 MPI.Send 和 MPI.Recv! 实现这一点，因为我们处理的是行分区，这样做更容易。然而，我们也可以用 MPI.Gather! 来完成。



```

function collect_result!(C,myC,comm)
    L = size(myC,1)
    rank = MPI.Comm_rank(comm)
    P = MPI.Comm_size(comm)
    if rank == 0
        lb = L*rank+1
        ub = L*(rank+1)
        C[lb:ub,:] = myC
        for source in 1:(P-1)
            lb = L*source+1
            ub = L*(source+1)
            MPI.Recv!(view(C,lb:ub,:),comm;source)
        end
    end
end

```

```

else
    dest = 0
    MPI.Send(myC,comm;dest)
end
C
end

```

4.4.5 运行和测试代码

在下面的代码中，我们运行并行代码并将其与串行代码进行比较。注意，我们只能在 rank0 上比较两个结果，因为只有它包含并行代码的结果。我们还包含了一个函数，用于生成给定大小 N 的随机距离表。

```

using MPI
MPI.Init()
floyd_mpi!(C,comm)
distribute_input(C,comm)
floyd_iterations!(myC,comm)
collect_result!(C,myC,comm)
function input_distance_table(n)
    threshold = 0.1
    mincost = 3
    maxcost = 9
    inf = 10000
    C = fill(inf,n,n)
    for j in 1:n
        for i in 1:n
            if rand() > threshold
                C[i,j] = rand(mincost:maxcost)
            end
        end
        C[j,j] = 0
    end
    C
end
function floyd!(C)
    n = size(C,1)
    @assert size(C,2) == n
    for k in 1:n
        for j in 1:n
            for i in 1:n
                @inbounds C[i,j] = min(C[i,j],C[i,k]+C[k,j])
            end
        end
    end
end

```

```

    end
    C
end
comm = MPI.Comm_dup(MPI.COMM_WORLD)
rank = MPI.Comm_rank(comm)
if rank == 0
    N = 24
else
    N = 0
end
C = input_distance_table(N)
C_par = copy(C)
floyd_mpi!(C_par, comm)
if rank == 0
    C_seq = copy(C)
    floyd!(C_seq)
    if C_seq == C_par
        println("Test passed!")
    else
        println("Test failed")
    end
end
end
run(`$(mpiexec()) -np 3 julia --project=. -e $code`)

```

运行此版本的代码。可能会发现，对于小规模问题和特定的 MPI 实现/网络状况，结果可能是正确的。但这并不能保证其在所有情况下的正确性。

4.5 识别和理解同步问题

在上面的代码中，并行代码的结果可能与串行代码的结果相同。然而，这足以断言代码是正确的吗？不幸的是，并非如此。

事实上，我们实现的并行代码是不正确的！无法保证此代码计算出正确的结果。

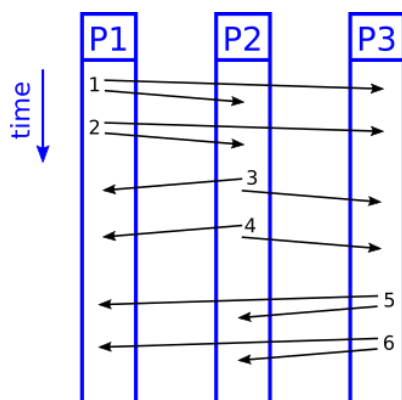
原因如下：

在 MPI 中，点对点消息在给定的发送者和接收者之间是不会超车的。假设进程 1 向进程 3 发送多条消息。所有这些消息将以 FIFO（先进先出）顺序到达。这是根据 MPI 标准 4.0 的 3.5 节。不幸的是，在我们的情况下这还不够。来自不同发送者的消息可能以错误的顺序到达。

核心问题: MPI 保证一对特定发送者和接收者之间的消息是按序到达的(FIFO)。但是，它不保证来自 不同 发送者的消息到达同一个接收者的顺序。

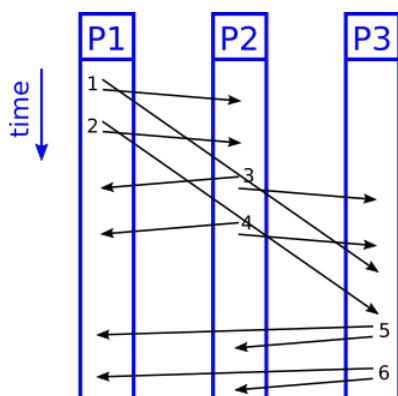
如果进程 1 向进程 3 发送消息，然后进程 2 向进程 3 发送其他消息，则不能保证进程 3 将首先接收来自进程 1 的消息，然后接收来自进程 2 的消息（见下图）。

如果我们幸运的话，所有消息都会按顺序到达。在我们的并行代码中，所有处理器将首先接收第 1 行，然后是第 2 行，然后是第 3 行，依此类推。计算结果将是正确的。



然而，处理器对之间的 FIFO 顺序不足以保证行按连续顺序到达。下图显示了一个反例。在这种情况下，由于某些未知原因，进程 1 和进程 3 之间的通信特别慢。结果，处理器 3 首先接收来自处理器 2 的消息，即使处理器 1 先发送了消息。

在我们的并行代码中，接收到的行将首先是第 3 行，然后是第 4 行，然后是第 1 行，然后是第 2 行，这是不正确的。但请注意，进程 3 以正确的顺序接收了来自进程 1 的所有消息（由 MPI 保证）。但在我们的算法中这还不够。



讨论:

- 为什么这个问题在小规模测试中可能不出现? (通信量小, 网络延迟差异不明显, 消息可能碰巧按序到达)。
- 如果使用了 `MPI.ANY_SOURCE` 来接收 `C_k`, 如何检测收到的行是哪一行? (可以通过 `MPI.Status` 对象获取发送者和标签, 然后在接收端进行缓冲和排序, 但这会增加复杂性)。

4.6 实现修正的并行算法 (使用 `MPI_Bcast!`)

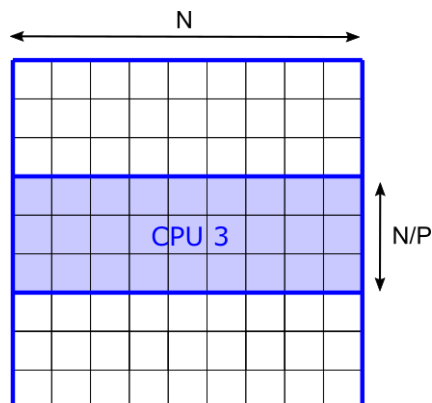
对于这种同步问题有几种解决方案:

1. 同步发送: 使用同步发送 `MPI_Ssend`。这种效率较低, 因为我们需要花费时间等待每条消息被接收。注意, 上面使用的阻塞发送 `MPI_Send` 并不能保证消息已被接收。
2. `MPI.Barrier`: 在每次 `k` 迭代结束时使用一个屏障。这很容易实现, 但会产生同步开销。
3. 对传入消息排序: 接收方对传入的消息进行排序, 例如使用 `MPI.Status` 来获取发送方 `rank`。这需要缓冲和额外的用户代码。
4. `MPI.Bcast!`: 使用 `MPI.Bcast!` 通信第 `k` 行。我们需要知道其他 `rank` 拥有哪些行, 因为我们不能在 `MPI.Bcast!` 中使用 `MPI.ANY_SOURCE`。然而, 如果行数是 `rank` 数量的倍数, 这就很简单了。

4.7 性能评估与分析

4.7.1 计算复杂度

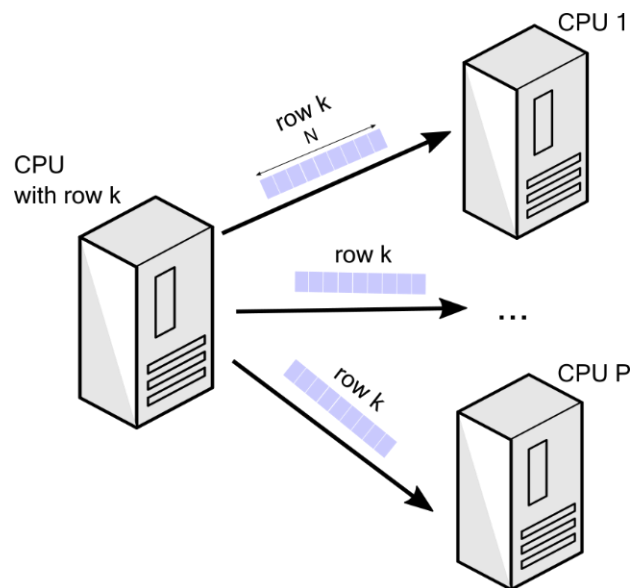
每个进程每次迭代更新 N^2/P 个条目。每次迭代的计算复杂度是 $O(N^2/P)$ 。



4.7.2 通信复杂度

每个迭代中，一个进程向 $P-1$ 个进程广播长度为 N 的消息。因此，每次迭代的发送成本是 $O(NP)$ (如果考虑点对点发送的总和，或者是广播操作的某种模型成本)。(更精确地说，对于广播，发送者只发送一次，但若用 $P-1$ 次点对点 `send` 模拟，则是 $(P-1)N$ 。若考虑一个进程广播到 $P-1$ 个进程，发送端工作量是 N ，接收端是 N 。)

$P-1$ 个进程在每次迭代中接收一个长度为 N 的消息。因此，每个进程每次迭代的接收成本是 $O(N)$ 。



通信与计算成本之比为：

- 发送端： $O(NP) / O(N^2/P) = O(P^2/N)$ (此处若发送成本理解为广播的源头发起成本是 $O(N)$)
- 接收端： $O(N) / O(N^2/P) = O(P/N)$

总结一下，发送/计算比率为 $O(P^2/N)$ ，接收/计算比率为 $O(P/N)$ 。如果 $P^2 \ll N$ ，该算法具有潜在的可扩展性。注意，这比矩阵-矩阵乘法（当 $P \ll N$ 时可扩展）要差。也就是说，当前算法比矩阵-矩阵乘法需要更大的问题规模。

4.7.3 性能测试

1. 计时

确保在并行代码的关键部分（主要是 `floyd_iterations!`）使用 `MPI.Wtime()` 进行计时。只在 0 号进程打印总时间，以避免屏幕输出混乱。

2. 测试不同规模

固定进程数 P ，改变问题规模 N ：例如，使用 $P=4$ 个进程，测试 $N = 240, 480, 960, \dots$ （确保 N 是 P 的倍数）。记录每次的执行时间。

固定问题规模 N ，改变进程数 P ：例如，使用 $N=960$ ，测试 $P = 1$ (串行), 2, 4, 8, ...。记录执行时间。

3. 计算指标

加速比 (Speedup): $S(P) = T_{\text{serial}} / T_{\text{parallel}}(P)$ ，其中 T_{serial} 是高效串行版本的执行时间， $T_{\text{parallel}}(P)$ 是使用 P 个进程的并行版本执行时间。

效率 (Efficiency): $E(P) = S(P) / P$ 。

4. 数据分析与绘图

绘制执行时间随 N 和 P 变化的曲线。绘制加速比和效率随 P 变化的曲线。

5. 分析与讨论

该并行算法是否展现了良好的加速效果？

效率如何随进程数增加而变化？为什么？（通常会下降，因为通信开销占比增加）。

比较 `floyd_iterations_v1!` (如果能稳定运行并得到正确结果的话，可能需要加入 `MPI.Barrier` 强制同步) 和 `floyd_iterations_v2!` (使用 `MPI.Bcast!`) 的性能。`MPI.Bcast!` 虽然正确性更好，但它是一个涉及所有进程的集体操作，其开销如何？

根据分析的通信复杂度 $O(NP)$ (发送端总开销) 或 $O(N)$ (每个接收端开销) 以及计算复杂度 $O(N^2/P)$ ，讨论实验结果是否与理论分析 $O(P^2/N)$ (发送/计算比) 或 $O(P/N)$ (接收/计算比) 的可扩展性预测相符。算法在 $P^2 \ll N$ 时才具有较好的可扩展性。

5、总结

本教学案例带领学生基于 `MWORKS.Syslab` 完整地经历了一个并行算法的设计、实现、调试和优化过程。通过 `Floyd-Warshall` 算法的 `MPI` 并行化实践，学生不仅接触了 `MWORKS.Syslab`，掌握了相关的并行编程技能，更能深刻体会到并行计算中数据依赖、通信、同步等核心概念的实际影响。通过对不同实现策略的比较和性能分析，学生将学会如何科学地评估和改进并行程序，为未来解决更复杂的工程与科研计算问题打下坚实基础。

5. 案例教学建议

(1) 案例引入：从现实世界中对最短路径查询的需求出发（如导航、网络路由），引出全源最短路径问题及其挑战，进而介绍 Floyd-Warshall 算法和并行计算的必要性。

(2) 课时分配（建议）

- 理论讲解（ASP 问题、Floyd 串行算法、并行计算基础、MPI 入门）：2-3 学时。
- 并行化策略设计与数据依赖分析：1-2 学时。
- MPI 编程实践（环境搭建、代码编写与调试）：4-6 学时。可采取分步实现的方式，从串行版本 → 初步并行版本 → 修正同步问题版本。
- 性能测试与分析：2-3 学时。
- 讨论与总结：1 学时。

(3) 教学讲授方式

- 理论讲授与引导：教师讲解核心概念和算法原理。
- 编程实践：学生在 MWORKS.Syslab 环境下动手编写和调试 MPI 程序。
- 小组讨论：针对并行化策略、数据依赖、同步问题、性能瓶颈等进行分组讨论。
- 演示与分析：教师或优秀学生演示代码和性能测试结果，引导分析。

(4) 课堂讨论总结：总结案例的关键知识点，讨论不同并行策略的优劣，以及并行程序设计的一般性原则和挑战。

6. 开放性问题

(1) 开放性问题

1. 除了 MPI.Bcast!，还有哪些同步策略可以用于解决 Floyd 并行算法中的数据分发问题？例如，使用同步发送 MPI_Ssend 或在每次 k 迭代后使用 MPI_Barrier。它们的优缺点分别是什么？

2. 本案例采用行划分。如果采用列划分或块划分（棋盘划分），数据依赖和通信模式会如何变化？性能可能会有何不同？

3. 在当前的行划分策略中，如果矩阵的行数 N 不能被进程数 P 整除，如何有效地处理负载均衡问题？

4. 是否可以利用非阻塞通信（如 `MPI_Isend`, `MPI_Irecv`, `MPI_Ibcast`）来重叠计算和通信，以进一步提升性能？如何实现？

5. 如何将本案例中的 Floyd 并行算法扩展到分布式内存集群上运行，并考虑更大规模的问题？

6. MWORKS.Syslab 还提供了哪些其他功能或工具箱，可以辅助并行程序的开发与调试？

（2）作业

修改 `floyd_iterations!` 函数，以保证计算结果的正确性。使用 `MPI.Bcast!` 来解决同步问题。注意：只在 `floyd_iterations!` 中使用 `MPI.Bcast!`，不要使用其他 MPI 指令。可以假设行数是进程数的整数倍。

7. 附件材料

案例中包含了详细的实验步骤和方案，另有案例配套的讲授 PPT、讲授视频和程序代码可供参考。

参考资料列表：

[1] Programming large-scale parallel systems - All pairs of shortest paths,

https://github.com/fverdugo/XM_40017/blob/main/notebooks/asp.ipynb

[2] MWORKS.Syslab, <https://www.tongyuan.cc/product/MWorksSyslab>